

Vigil Protocol

Build Specification — BOT Chain Builder Challenge #2 (AI × RWA)

Version 1.0 — Prepared for a single build pass with Claude Code

Vigil is a continuous-truth oracle for real-world assets, paired with two autonomous financial primitives that act on that truth without human intervention:

- **Attestation Engine** — an AI agent that reads physical evidence (images, documents, sensor data) or live business revenue data and produces signed, confidence-scored, on-chain attestations.
- **Collateral Vault** — reads asset attestations and autonomously executes margin calls / liquidations when collateral health degrades.
- **Revenue Bond Vault** — reads revenue attestations and autonomously mints, prices, and repays revenue-share financing in real time.

One oracle, two consumers. This document is the complete build package: requirements, architecture, exact SDKs/APIs, smart contract specs, and a phased set of prompts to hand directly to Claude Code.

1. Executive Summary

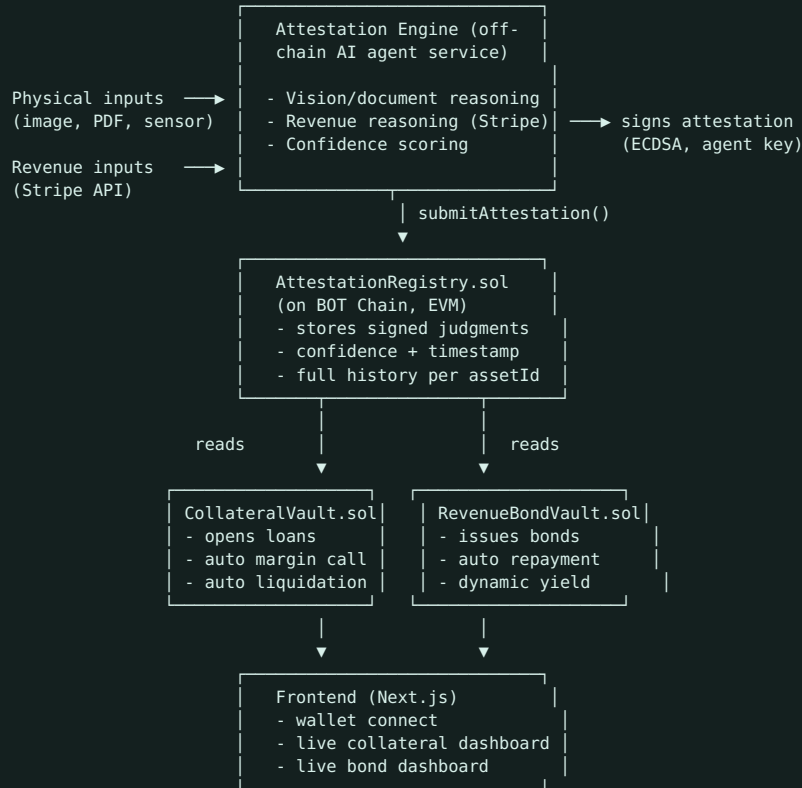
What we're building: A platform, not a single-purpose app. The core deliverable is `AttestationRegistry.sol` — a generalized on-chain registry that stores AI-signed, confidence-scored judgments about real-world state (physical or financial). Two consumer contracts subscribe to it: one for physical collateral, one for revenue-backed financing. The AI is the decision-maker in both cases — it is not a chatbot bolted onto a dashboard, it is the thing that causes on-chain transactions to happen.

Why this fits BOT Chain Builder Challenge #2 specifically:

Challenge requirement (from official rules)	How Vigil satisfies it
AI must be a core decision-making capability , not auxiliary chat/API wrapper	AI attestations directly trigger vault state changes (liquidation, bond issuance, repayment)
RWA track: asset management, restaking, product aggregation, infrastructure	Attestation Engine is infrastructure; Collateral Vault is asset management
Must not significantly overlap with a "wallet, dashboard, marketplace, DEX, bridge, chatbot, explorer, CRUD app"	Vigil is none of these — it's an oracle + two financial primitives
Mainnet deployment, wallet integration, GitHub repo, public demo, complete business workflow	Directly scoped into Phase 6-7 below

Build effort allocation (2-week window): ~50% Attestation Engine + Registry (this is the real IP — make it excellent), ~35% Collateral Vault (your flagship demo), ~15% Revenue Bond Vault (proves the registry is reusable — lighter feature set is fine).

2. Architecture Overview



Design principle: the registry knows nothing about vaults. Vaults know nothing about each other. This is what makes the "one AI engine, two products" pitch technically true rather than just a slide.

3. BOT Chain Network Reference

Verified against BOT Chain's official site and ChainList as of this document's preparation. **Confirm the RPC URL yourself** at the wallet "Add Network" flow or the dev docs quick-guide before Phase 0 — it wasn't exposed on the public pages this document was built from.

Field	Value
Network name	BOT Chain Mainnet
Chain ID	677 (hex 0x2A5)
Native currency	BOT
Type	EVM-compatible L1, PoSA/SPoA hybrid consensus
Block time	~0.75s
Finality	~0.9s average (fast finality, ~2s max)
Block explorer	https://scan.botchain.ai/
Official wallet	https://wallet.botchain.ai/
Bridge	https://bridge.botchain.ai/
DEX	https://dex.botchain.ai/
Testnet faucet	https://faucet.botchain.ai/basic/
Developer docs	https://dev-docs.botchain.ai/
GitHub org	https://github.com/BOTChain-bot
Ecosystem support	https://www.botchain.ai/ecosystem-support

Action item for Phase 0: open `wallet.botchain.ai` , use "Add Network," and copy the exact RPC URL it inserts (or pull it from the dev docs quick-guide linked above) into your `.env` . Do not guess this value — an incorrect RPC will silently fail every later phase.

4. Full Tech Stack & SDKs/APIs

Layer	Choice	Why
Smart contracts	Solidity ^0.8.24, Hardhat (TypeScript)	Best plugin support for verify/deploy scripting, keeps whole repo in TS
Contract libraries	OpenZeppelin Contracts (<code>AccessControl</code> , <code>ECDSA</code> , <code>ERC20</code>)	Battle-tested, avoids reinventing signature verification
Chain interaction (backend/agent)	<code>ethers.js</code> v6	Simplest signing + contract-call ergonomics for a Node agent
Chain interaction (frontend)	<code>viem</code> + <code>wagmi</code> v2	Modern React hooks for wallet connect and contract reads/writes
Wallet connect UI	<code>@rainbow-me/rainbowkit</code> (or <code>ConnectKit</code>)	Fast to wire up, supports MetaMask/WalletConnect out of the box
Frontend framework	Next.js 14 (App Router) + TypeScript + Tailwind	Fast to ship, easy Vercel deploy for the public demo URL
AI reasoning	Anthropic Claude API — <code>claude-sonnet-5</code> (confirm current vision-capable model at docs.claude.com)	Multimodal (image+text) reasoning with strict JSON-mode output; same vendor as the coding agent building this, no extra account needed
Revenue data source	Stripe API, test mode	Real, working revenue data without needing a live business; <code>balance</code> , <code>charges.list</code> , <code>balance_transactions</code> endpoints
Off-chain agent runtime	Node.js 20 + TypeScript, <code>express</code> for a small control API	Keeps agent, contract calls, and API triggers in one deployable service
Test stablecoin	Self-deployed <code>MockUSDT.sol</code> (ERC20, 6 decimals, owner-mintable)	BOT Chain testnet may not have a bridged stablecoin available; label clearly as a demo stand-in in the UI
Testing	Hardhat + Chai/Mocha, <code>hardhat-gas-reporter</code>	Standard, fast
Deployment target	BOT Chain Mainnet (required for award eligibility — testnet-only does not qualify)	Per official challenge rules

5. Smart Contract Specifications

5.1 `AttestationRegistry.sol`

Purpose: generalized, append-only store of signed AI judgments about any `assetId` .

```

struct Attestation {
    bytes32 assetId;          // keccak256 of a stable asset identifier
    uint8  assetType;         // 0 = Physical, 1 = Revenue
    int256 value;             // valuation (physical) or repayment amount (revenue), scaled 1e18
    uint16 confidenceBps;     // 0-10000 (basis points)
    bytes32 dataHash;         // hash of the underlying evidence (image/doc/API payload)
    uint64 timestamp;
    address signer;
}

// core functions
function submitAttestation(
    bytes32 assetId,
    uint8 assetType,
    int256 value,
    uint16 confidenceBps,
    bytes32 dataHash,
    bytes calldata signature
) external; // verifies signer has ORACLE_ROLE via ECDSA.recover over the packed payload

function getLatest(bytes32 assetId) external view returns (Attestation memory);
function getHistory(bytes32 assetId) external view returns (Attestation[] memory);
function isStale(bytes32 assetId, uint64 maxAgeSeconds) external view returns (bool);

// access control
// DEFAULT_ADMIN_ROLE -> deployer / project multisig
// ORACLE_ROLE -> the Attestation Engine's signing address(es)

// events
event AttestationSubmitted(bytes32 indexed assetId, uint8 assetType, int256 value, uint16 confidenceBps, address indexed signer, uint64 timestamp);

```

Use OpenZeppelin `AccessControl` for role management. Signature scheme: agent signs

`keccak256(abi.encodePacked(assetId, assetType, value, confidenceBps, dataHash, timestamp))`; contract recovers and checks `hasRole(ORACLE_ROLE, recovered)`.

5.2 CollateralVault.sol

Purpose: open loans against attested physical/document-backed collateral; autonomously act when attested value drops.

```

struct Loan {
    address borrower;
    bytes32 assetId;
    uint256 principal;        // in MockUSDT, 6 decimals
    uint16 liquidationThresholdBps; // e.g. 8000 = liquidate above 80% LTV
    bool active;
}

function openLoan(bytes32 assetId, uint256 principal, uint16 liquidationThresholdBps) external returns (uint256 loanId);
// requires: registry.getLatest(assetId).confidenceBps >= MIN_CONFIDENCE
// requires: computed LTV at open time is healthy

function currentLTV(uint256 loanId) public view returns (uint256 ltvBps);
// = principal * 10000 / registry.getLatest(assetId).value

function checkAndLiquidate(uint256 loanId) external;
// public/keeper-callable - if currentLTV > liquidationThresholdBps, mark defaulted,
// emit Liquidated, seize/freeze position

function repay(uint256 loanId, uint256 amount) external;

event LoanOpened(uint256 indexed loanId, address indexed borrower, bytes32 assetId, uint256 principal);
event MarginCall(uint256 indexed loanId, uint256 ltvBps);
event Liquidated(uint256 indexed loanId, uint256 ltvBps, uint64 timestamp);
event Repaid(uint256 indexed loanId, uint256 amount, uint256 remaining);

```

5.3 RevenueBondVault.sol

Purpose: issue revenue-share financing against attested revenue data; autonomously settle repayment as new revenue attestations arrive.

```

struct Bond {
    address issuer;           // the business/creator being financed
    bytes32 businessId;
    uint256 principalFunded;
    uint256 outstandingBalance;
    uint16 revenueShareBps;    // e.g. 500 = 5% of settled revenue goes to repayment
    bool active;
}

function issueBond(bytes32 businessId, uint256 principal, uint16 revenueShareBps) external returns (uint256 bondId);
    // requires: registry.getLatest(businessId).assetType == Revenue
    // requires: registry.getLatest(businessId).confidenceBps >= MIN_CONFIDENCE
    // transfers `principal` MockUSDT to the issuer

function settleRevenue(uint256 bondId) external;
    // reads registry.getLatest(businessId) (assetType Revenue, value = new period revenue)
    // repayment = value * revenueShareBps / 10000
    // transfers repayment from a funded liquidity pool to bond, reduces outstandingBalance

event BondIssued(uint256 indexed bondId, address indexed issuer, uint256 principal, uint16 revenueShareBps);
event RevenueSettled(uint256 indexed bondId, uint256 repaymentAmount, uint256 outstandingBalance);
event BondRepaidInFull(uint256 indexed bondId);

```

5.4 MockUSDT.sol

Standard OpenZeppelin ERC20, 6 decimals, `mint(address,uint256)` restricted to deployer, used purely so the demo has a working settlement asset on a chain without a confirmed bridged stablecoin. Label this explicitly in the UI ("Demo settlement token — testnet stand-in for USDT") so judges don't mistake it for a real asset claim.

6. AI Attestation Engine Specification

A Node/TypeScript service (`/agent`) with two ingestion pipelines feeding one canonical output shape.

6.1 Physical asset pipeline

- Input: image (base64) + optional supporting document text
- Call Claude API with vision content blocks + a system prompt instructing **strict JSON-only output**, temperature 0
- Output shape:

```

{
  "exists": true,
  "condition_score": 0,
  "estimated_value_usd": 0,
  "confidence": 0,
  "reasoning": "short justification, not shown on-chain, logged off-chain for audit"
}

```

- Agent maps `estimated_value_usd` → `value` (scaled), `confidence` → `confidenceBps`, hashes the raw image+doc payload → `dataHash`, signs, calls `submitAttestation`.

6.2 Revenue pipeline

- Input: Stripe test-mode API key (per connected business)
- Pull `balance_transactions.list` and recent `charges.list`, compute trailing revenue + volatility
- Call Claude API with the computed stats (not raw PII) to reason over risk and recommend terms
- Output shape:

```
{
  "period_revenue_usd": 0,
  "volatility_score": 0,
  "risk_score": 0,
  "recommended_revenue_share_bps": 0,
  "confidence": 0
}
```

- Same signing/submission path as above, `assetType = 1` (Revenue) .

6.3 Prompt design notes for Claude Code to implement

- Use the Messages API with a `system` prompt that forbids any output outside a single JSON object (no markdown fences, no prose) — parse defensively anyway (strip code fences before `JSON.parse`).
- Keep confidence calibrated: don't let the model default to 95%+ on thin evidence — instruct it explicitly to lower confidence when the document/image is ambiguous, low-resolution, or the revenue history is short.
- Log every raw model response off-chain (for the audit trail / judge Q&A) even though only the structured summary goes on-chain.

7. Frontend Specification

Two dashboards sharing one wallet-connect shell:

- **Collateral view:** list of open loans, live LTV bar per loan (color-coded), "Simulate new attestation" demo button (calls the agent's `/attest/physical` endpoint), on-chain liquidation event feed.
- **Bond view:** "Connect Stripe (test mode)" flow, live bond terms once underwritten, "Simulate revenue period" demo button (calls `/attest/revenue`), repayment history feed pulled from `RevenueSettled` events.
- Wallet connect via RainbowKit, network guard that prompts switching to BOT Chain (chain ID 677) if the connected wallet is elsewhere.
- Every on-chain action links out to `scan.botchain.ai` for the transaction — this matters for judge verification.

8. Functional Requirements (mapped to official award-eligibility rules)

- ☐ Successfully deployed to **BOT Chain Mainnet** (not testnet-only)
- ☐ Publicly verifiable product with a complete user/business operation loop (consumer-ready)
- ☐ Supports wallet connection and completes the full core business flow end-to-end
- ☐ Publicly accessible website or online demo (deployed frontend URL)
- ☐ GitHub repository submitted (private is fine, but judges need access)
- ☐ Demo video recorded (recommended, not strictly required)
- ☐ Project is original — not a renamed/redeployed prior submission

9. Non-Functional Requirements

- Agent signing key must be a **dedicated hot wallet**, funded only with the minimum BOT needed for gas — never reuse a personal/treasury key.
- All contract state changes triggered by attestations must be **publicly explainable**: emit enough event data that a judge reading the explorer can reconstruct why a liquidation or repayment happened.
- Confidence thresholds (`MIN_CONFIDENCE`) must be enforced on-chain, not just suggested off-chain — this is what makes the AI's judgment load-bearing rather than decorative.

- Keep the MVP scope honest: two demo flows fully working beats three flows half-working.

10. Demo Script (the "wow" sequence)

1. **Open with the registry, not the app.** Show `AttestationRegistry` on `scan.botchain.ai` — empty, or with one prior entry. Explain in one sentence: "This is the only piece of state both products share."
2. **Act 1 — Physical collateral.** Show a physical object on camera → hit "Simulate attestation" → within seconds a signed attestation lands on-chain → `CollateralVault` shows healthy LTV. Now feed a "damaged/lost" scenario → new attestation posts → LTV crosses threshold live → `checkAndLiquidate` fires → judges watch the liquidation transaction land in the explorer in real time.
3. **Act 2 — Revenue financing.** Connect a live Stripe test account → agent underwrites in seconds → bond issued → stablecoin lands in a wallet on screen. Simulate a revenue spike → dashboard recomputes the next settlement → `settleRevenue` fires on-chain automatically.
4. **Close on the platform claim.** "Same registry, same AI reasoning pattern, two completely different financial products — built once." Pull the explorer back up and show both vaults' events interleaved on the same registry contract.

11. Build Plan — Phased Prompts for Claude Code

Paste each phase's prompt into Claude Code **in order**. Don't start a phase until the previous one's "Definition of Done" is satisfied — each phase assumes the prior phase's files exist.

Phase 0 — Environment & Scaffolding

Goal: repo skeleton, BOT Chain network config, wallets funded, nothing deployed yet.

PROMPT FOR CLAUDE CODE — PHASE 0

Set up a new monorepo called "vigil-protocol" with three workspaces:

```
/contracts  - Hardhat + TypeScript project
/agent      - Node.js + TypeScript service (Express)
/frontend   - Next.js 14 (App Router) + TypeScript + Tailwind
```

Root-level: pnpm workspaces, a single `.env.example` listing every variable needed across all three workspaces (do NOT put real secrets in it):

```
BOT_CHAIN_RPC_URL=
BOT_CHAIN_CHAIN_ID=677
DEPLOYER_PRIVATE_KEY=
ORACLE_SIGNER_PRIVATE_KEY=
ANTHROPIC_API_KEY=
STRIPE_TEST_SECRET_KEY=
NEXT_PUBLIC_WALLETCONNECT_PROJECT_ID=
```

In `/contracts`, configure `hardhat.config.ts` with a "botchain" network entry using `chainId 677` and `BOT_CHAIN_RPC_URL` from env. Add a "botchain-testnet" network placeholder too (same shape, different RPC) so we can rehearse deployments before mainnet.

Do not write any contract logic yet. Just get ``pnpm install`` and ``npx hardhat compile`` (on an empty `contracts` dir) working cleanly in each workspace, and confirm ``npx hardhat run scripts/checkNetwork.ts --network botchain`` can read the current block number from BOT Chain.

Definition of done: I can run one command from repo root that installs everything, and a network-check script proves we can talk to BOT Chain.

Definition of Done: `checkNetwork.ts` prints a real block number from BOT Chain. If this fails, stop — the RPC URL from Section 3 is wrong, don't proceed.

Phase 1 — Smart Contracts + Tests

PROMPT FOR CLAUDE CODE — PHASE 1

In /contracts, implement three Solidity ^0.8.24 contracts using OpenZeppelin Contracts (AccessControl, ECDSA, ERC20):

1. AttestationRegistry.sol — exact spec below.
2. CollateralVault.sol — exact spec below.
3. RevenueBondVault.sol — exact spec below.
4. MockUSDT.sol — standard ERC20, 6 decimals, owner-only mint().

[paste the full Section 5 contract specs from the build doc here — struct fields, function signatures, events, and access control notes, exactly as written]

Requirements:

- AttestationRegistry uses AccessControl with ORACLE_ROLE for signers and DEFAULT_ADMIN_ROLE for the deployer. submitAttestation() must verify the ECDSA signature recovers to an address holding ORACLE_ROLE.
- CollateralVault and RevenueBondVault both read AttestationRegistry via an immutable reference set in their constructors.
- Enforce a MIN_CONFIDENCE constant (start at 7000 = 70%) on-chain in both vaults — reject actions if the latest attestation's confidenceBps is below this.
- Write full Hardhat test suites for all three contracts covering: happy path, unauthorized signer rejection, stale/low-confidence rejection, and the full liquidation and revenue-settlement flows using a mock signer wallet to simulate the Attestation Engine.
- Add a hardhat-gas-reporter config so we know deployment + call costs before touching mainnet.

Definition of done: `npx hardhat test` passes with all of the above scenarios covered, and gas report output is visible in the console.

Definition of Done: full test suite green, gas report visible.

Phase 2 — Testnet Deployment & Verification

PROMPT FOR CLAUDE CODE — PHASE 2

Write /contracts/scripts/deploy.ts that deploys, in order:
MockUSDT -> AttestationRegistry -> CollateralVault(registry, mockUSDT)
-> RevenueBondVault(registry, mockUSDT), grants ORACLE_ROLE on the registry to the ORACLE_SIGNER PRIVATE KEY's address, and writes all four resulting addresses + the deployment block number to /contracts/deployments/<network>.json.

Run this against the "botchain-testnet" network first using the testnet faucet (<https://faucet.botchain.ai/basic/>) for gas. Confirm each contract on <https://scan.botchain.ai/> (or the testnet explorer if separate) and paste the resulting contract addresses back into deployments/botchain-testnet.json.

Do not deploy to mainnet yet — that's Phase 7, after the full app is demo-ready end to end.

Definition of done: four verified contracts live on BOT Chain testnet, addresses saved to a deployments file the frontend and agent can both import from.

Definition of Done: deployments JSON populated with real testnet addresses, contracts visible/verified on the explorer.

Phase 3 — Attestation Engine (AI Agent)

PROMPT FOR CLAUDE CODE – PHASE 3

In /agent, build an Express service with two endpoints:

```
POST /attest/physical
body: { assetId: string, imageBase64: string, note?: string }
- Calls the Anthropic Claude API (model: claude-sonnet-5 – check docs.claude.com for the current vision-capable model id and update if this has changed) with the image as a vision content block and a system prompt that demands STRICT JSON output only, matching: { exists, condition_score, estimated_value_usd, confidence, reasoning }
- Defensively strip any markdown code fences before JSON.parse.
- Hash the raw image+note payload (keccak256) -> dataHash.
- Sign keccak256(abi.encodePacked(assetId, assetType=0, value, confidenceBps, dataHash, timestamp)) with ORACLE_SIGNER_PRIVATE_KEY using ethers v6.
- Call AttestationRegistry.submitAttestation(...) on BOT Chain testnet and return the resulting tx hash + parsed attestation to the caller.
```

```
POST /attest/revenue
body: { businessId: string, stripeTestSecretKey: string }
- Pulls recent balance_transactions and charges from Stripe (test mode).
- Computes trailing period revenue and a simple volatility measure (coefficient of variation over the last N periods).
- Calls Claude API with ONLY the computed statistics (never raw Stripe payloads) to produce:
  { period_revenue_usd, volatility_score, risk_score, recommended_revenue_share_bps, confidence }
- Same hash/sign/submit flow as above, with assetType=1.
```

Log every raw Claude response to /agent/logs/ for audit purposes even though only the structured summary goes on-chain.

Add a third endpoint, GET /health, that checks RPC connectivity and Anthropic/Stripe key validity without making billable calls.

Definition of done: I can curl both /attest endpoints with test payloads and see a real transaction land on BOT Chain testnet, visible on the explorer, with correctly decoded event data.

Definition of Done: both endpoints produce real on-chain attestations from real (test-mode) inputs.

Phase 4 — Frontend

PROMPT FOR CLAUDE CODE – PHASE 4

In /frontend, build a Next.js 14 App Router site using wagmi + viem + RainbowKit, configured for BOT Chain (chainId 677, using the same RPC from .env). Import contract addresses from /contracts/deployments/botchain-testnet.json.

Pages:

- / – landing page explaining Vigil in three short sections (Attestation Engine, Collateral Vault, Revenue Bond Vault), wallet connect button, a "Launch App" CTA
- /collateral – list of the connected wallet's open loans, a live LTV bar per loan (green/yellow/red), a "Simulate new attestation" button that posts to the agent's /attest/physical endpoint with a bundled demo image, and a live feed of LoanOpened/MarginCall/Liquidated events read via viem's watchContractEvent
- /revenue – "Connect Stripe (test mode)" flow (paste a Stripe test secret key into a form – label clearly as test-mode only), shows underwriting result once the agent responds, a "Simulate revenue period" button, and a live feed of BondIssued/RevenueSettled events

Add a network guard component that prompts the user to switch to BOT Chain if their wallet is on a different chain. Every on-chain event in both feeds should link out to <https://scan.botchain.ai/tx/<hash>>.

Definition of done: I can connect a wallet, trigger both simulate buttons, and watch both dashboards update within a few seconds of the on-chain transaction confirming – no manual page refresh needed.

Definition of Done: both dashboards live-update from real on-chain events without a refresh.

Phase 5 — Integration Pass

PROMPT FOR CLAUDE CODE – PHASE 5

Do a full end-to-end pass across /contracts, /agent, and /frontend:

1. Fix any mismatches between the deployed ABI and what the frontend/agent expect (regenerate typechain types in /contracts and re-import into both).
2. Add a seed script (/contracts/scripts/seedDemo.ts) that opens one sample loan and issues one sample bond against two pre-attested assetIds, so the demo never starts from a completely empty state.
3. Write a top-level README.md covering: architecture summary (reuse Section 2 of the build doc), setup instructions, how to run all three workspaces locally, and the demo script (reuse Section 10).
4. Add a CI-lite check: a single script that runs contract tests, type-checks the agent, and builds the frontend, so nothing is silently broken before demo day.

Definition of done: starting from a clean clone, `pnpm install` + one setup script gets a working local environment against BOT Chain testnet, seeded with one example of each flow.

Definition of Done: clean-clone setup works end to end against testnet.

Phase 6 — Demo Polish

PROMPT FOR CLAUDE CODE – PHASE 6

Polish pass, no new features:

1. Tighten the two "Simulate" flows so each completes in well under 30 seconds end to end (image attestation, revenue attestation) – add loading states that show each step (calling AI -> signing -> submitting on-chain -> confirmed) so the audience can follow along even during the wait.
2. Add a small "How this works" expandable panel on both dashboard pages summarizing the AI -> attestation -> on-chain action pipeline in plain language, for judges skimming quickly.
3. Record a 2–3 minute demo video following the Section 10 script, narrating what's happening at each step.
4. Deploy /frontend to Vercel and /agent to a small always-on host (Railway/Render/Fly.io) so the public demo URL works without my laptop running anything.

Definition of done: a judge can open the public URL cold, connect a wallet, and complete both demo flows without me present.

Definition of Done: public URL works standalone, cold, no local dependency.

Phase 7 — Mainnet Deployment & Submission

PROMPT FOR CLAUDE CODE – PHASE 7

Repeat Phase 2's deployment script against the "botchain" (mainnet) network instead of testnet. Before running:

- Confirm the deployer wallet is funded with real BOT for gas.
- Confirm ORACLE_SIGNER address is funded separately (small amount – it only needs enough for attestation submission gas).
- Double check MIN_CONFIDENCE and liquidationThresholdBps constants are the values we actually want live (not test-loosened values).

Deploy, verify all four contracts on <https://scan.botchain.ai/>, update /contracts/deployments/botchain.json, and repoint /frontend and /agent env vars at the mainnet deployment and mainnet RPC.

Re-run the full demo script end to end against mainnet exactly once before submission to confirm nothing behaves differently than testnet.

Finally, assemble the submission:

- Public frontend URL (mainnet-pointed)
- GitHub repo link (grant judge access if private)
- Demo video link
- One-paragraph project description emphasizing: AI as the core decision-making capability, the shared-registry platform architecture, and the two live consumer products

Definition of done: mainnet contracts verified and linked from the explorer, public demo works against mainnet, submission fields filled per the official Challenge #2 requirements.

Definition of Done: everything from Section 8's checklist is checked.

12. Submission Checklist (copy directly from official rules)

- ☐ BOT Chain **Mainnet** deployment (testnet-only is disqualifying)
- ☐ Public website or online demo, working cold with no local dependency
- ☐ Wallet integration completing the full core business flow
- ☐ GitHub repository (private access acceptable if judges are granted entry)
- ☐ Demo video (recommended)

- ☐ Project is original — not a rename/redeploy of a prior submission
 - ☐ Submission explains, if migrating any component from elsewhere: why BOT Chain, what's new in this version, and the plan to keep growing usage on-chain
-

13. Appendix — Reference Links

- BOT Chain Mainnet: <https://www.botchain.ai/>
 - Developer Docs: <https://dev-docs.botchain.ai/>
 - Block Explorer: <https://scan.botchain.ai/>
 - Bridge: <https://bridge.botchain.ai/>
 - BOT DEX: <https://dex.botchain.ai/>
 - BOT Wallet: <https://wallet.botchain.ai/>
 - Testnet Faucet: <https://faucet.botchain.ai/basic/>
 - GitHub: <https://github.com/BOTChain-bot>
 - Ecosystem Support: <https://www.botchain.ai/ecosystem-support>
 - Claude API docs (confirm current models/vision support before Phase 3): <https://docs.claude.com/en/api/overview>
 - OpenZeppelin Contracts: <https://docs.openzeppelin.com/contracts>
 - Hardhat: <https://hardhat.org/docs>
 - wagmi: <https://wagmi.sh>
 - viem: <https://viem.sh>
 - RainbowKit: <https://www.rainbowkit.com>
 - Stripe API (test mode): <https://docs.stripe.com/api>
-

End of build specification. Work phase by phase — don't let Claude Code skip ahead to Phase 7 early; the definition-of-done gates exist because a broken RPC config or a bad contract assumption compounds badly three phases later.